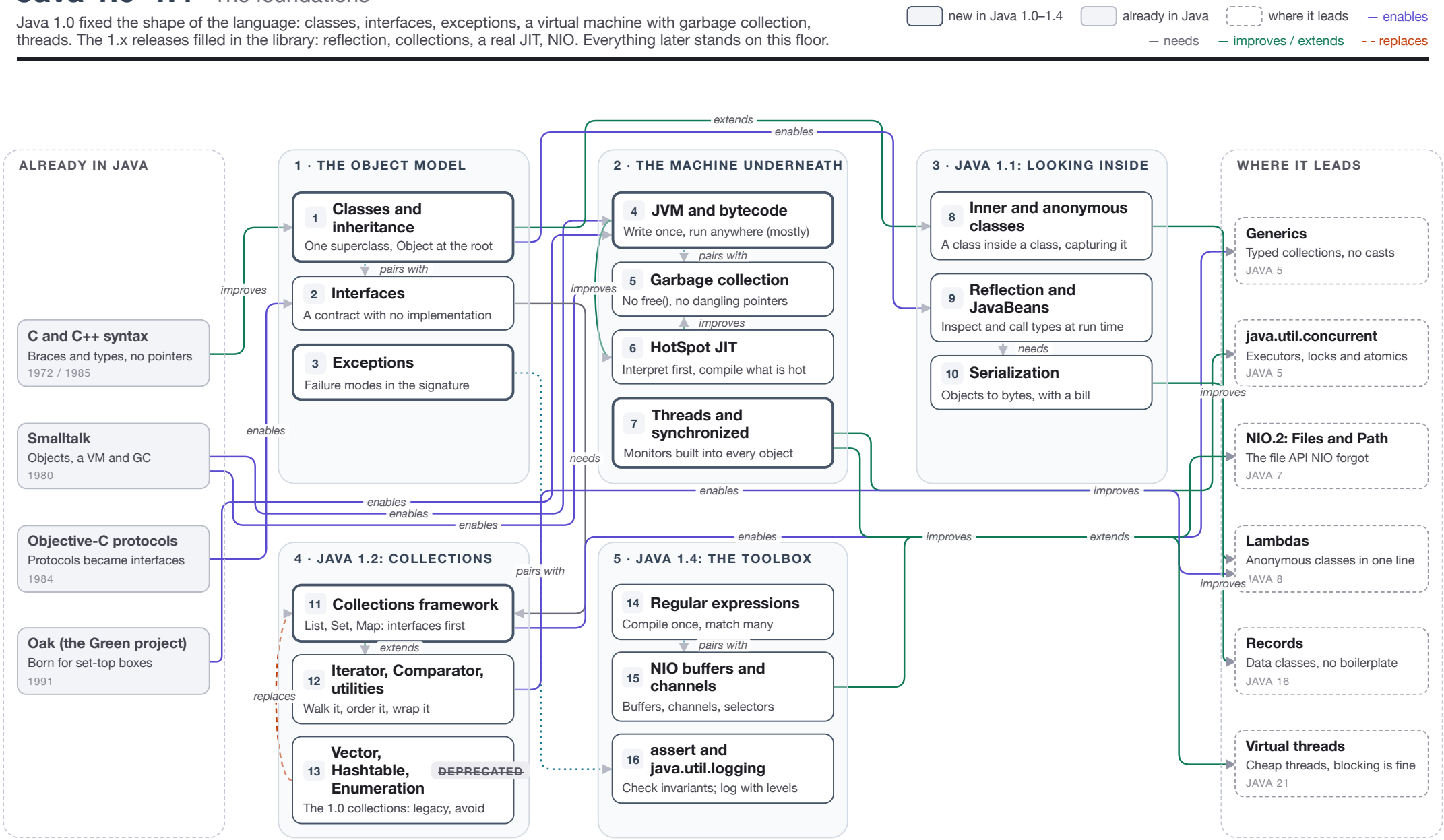
CHAPTER 1 · KNOWLEDGE GRAPH · JAVA 1.0 – 1.4 · 1996 – 2002

Java 1.0–1.4 · The foundations

Java 1.0 fixed the shape of the language: classes, interfaces, exceptions, a virtual machine with garbage collection, threads. The 1.x releases filled in the library: reflection, collections, a real JIT, NIO. Everything later stands on this floor.

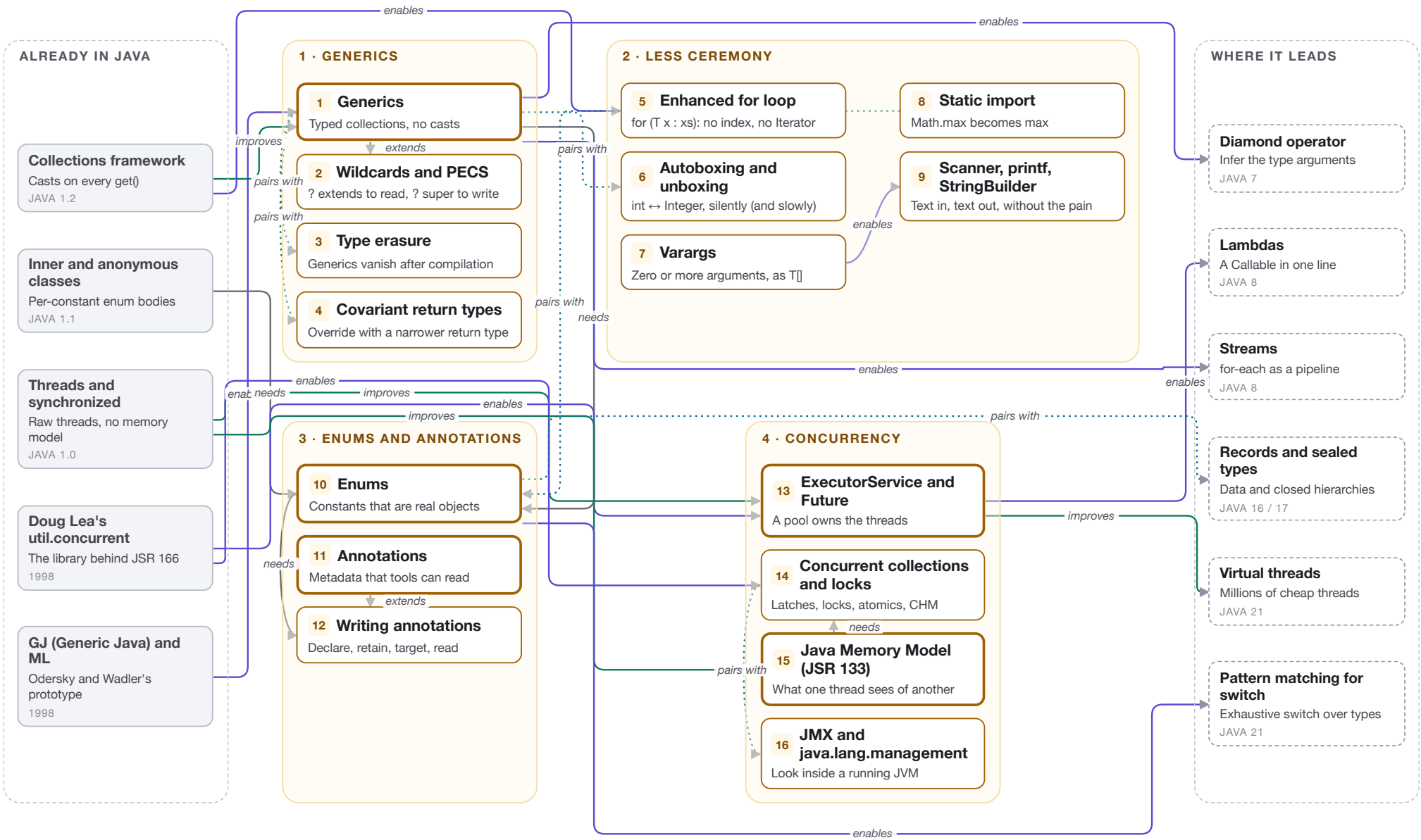


Read left to right: what Java already had → what Java 1.0–1.4 added, grouped by job (numbers match the cards and the cheat sheet) → what it made possible later.

Java 5 · The language grows up

One release, almost a new language: generics, enums, annotations, autoboxing, for-each, varargs, static imports, plus java.util.concurrent and a memory model that finally meant something. Most "modern Java" habits start here.

new in Java 5 · already in Java · where it leads · — enables · — needs · — improves / extends · - - replaces

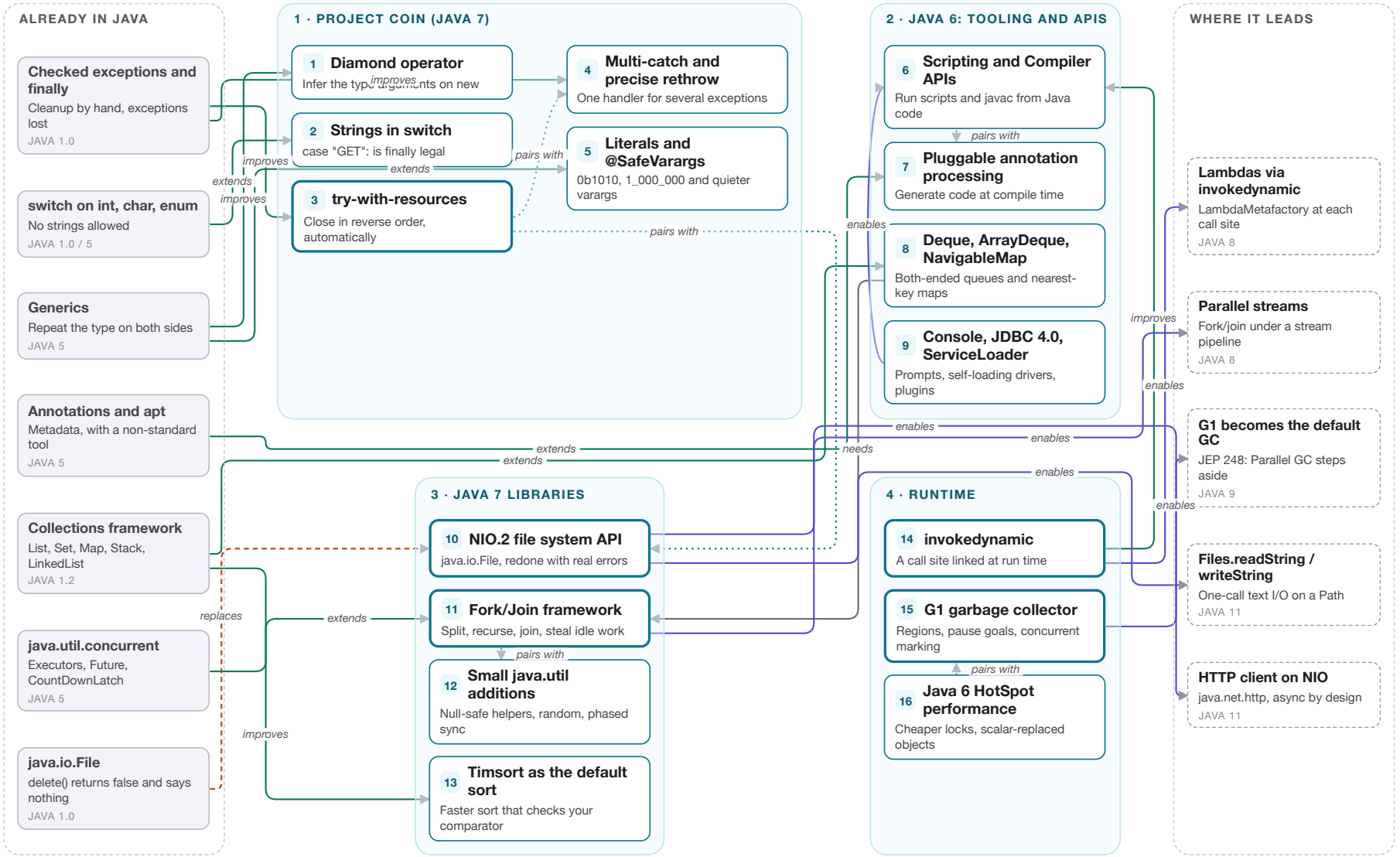


Read left to right: what Java already had → what Java 5 added, grouped by job (numbers match the cards and the cheat sheet) → what it made possible later.

Java 6–7 · Polish and plumbing

Mustang made the JDK programmable and faster; Dolphin, after five years, trimmed the syntax, fixed file I/O and laid the bytecode groundwork that lambdas would stand on three years later.

new in Java 6–7 · already in Java · where it leads · — enables · — needs · — improves / extends · - - replaces

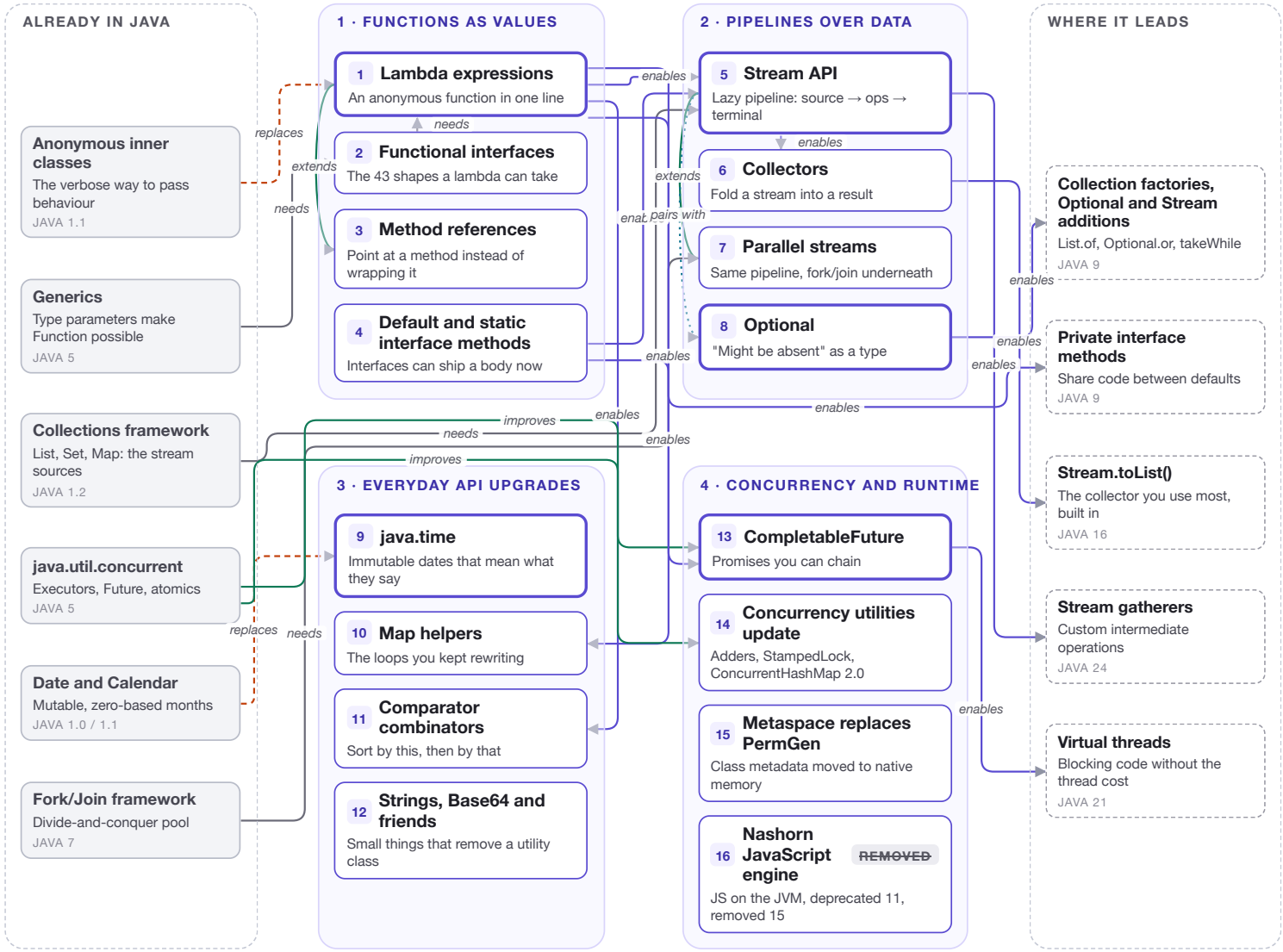


CHAPTER 4 · KNOWLEDGE GRAPH · JAVA 8 · MAR 2014 · LTS

Java 8 · The functional turn

Functions became values, collections became pipelines, and dates finally made sense. Java 8 is the baseline every later version assumes you know.

new in Java 8 · already in Java · where it leads · — enables · — needs · — improves / extends · - - replaces

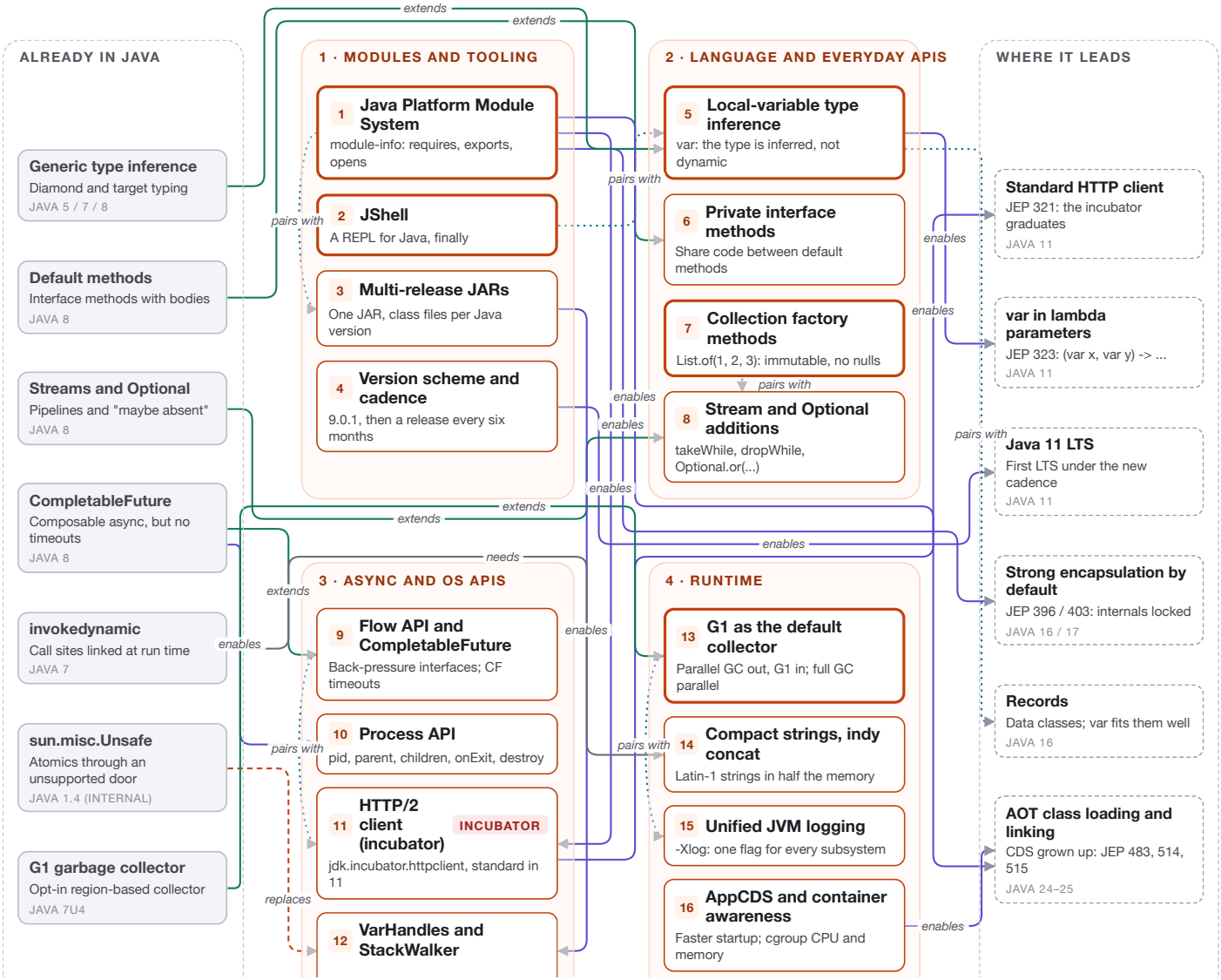


Read left to right: what Java already had → what Java 8 added, grouped by job (numbers match the cards and the cheat sheet) → what it made possible later.

Java 9–10 · Modules and the new cadence

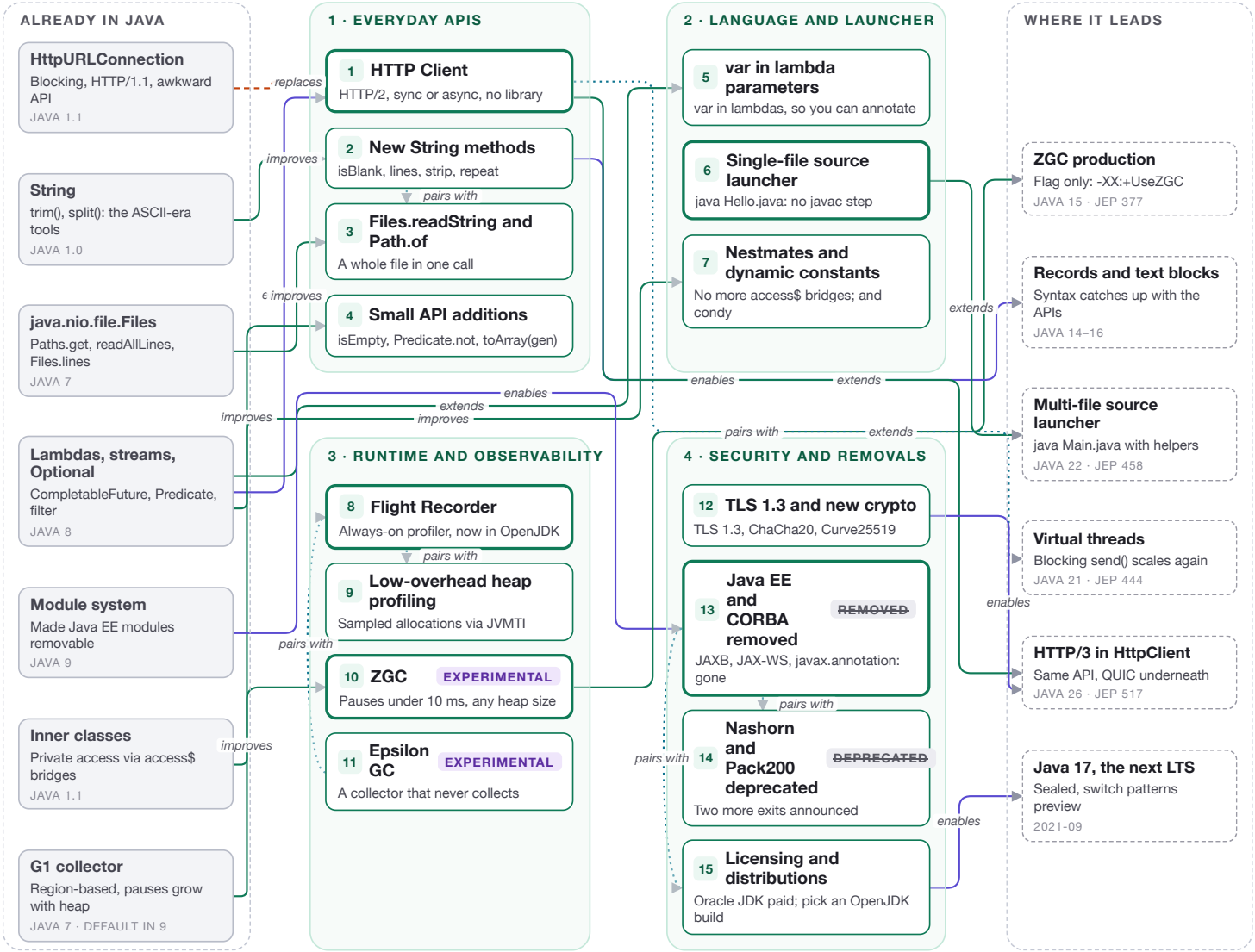
Java 9 reorganised the platform into modules and took three years doing it; Java 10 arrived six months later and changed how Java ships for good. In between: var, List.of, JShell and a long list of small library gifts.

new in Java 9–10 · already in Java · where it leads · — enables · — needs · — improves / extends · - - replaces



Java 11 · The new baseline

The first LTS of the six-month era. Few new keywords, but a real HTTP client, a scriptable launcher, two new collectors, and the removal of Java EE: the version every migration from 8 has to cross.

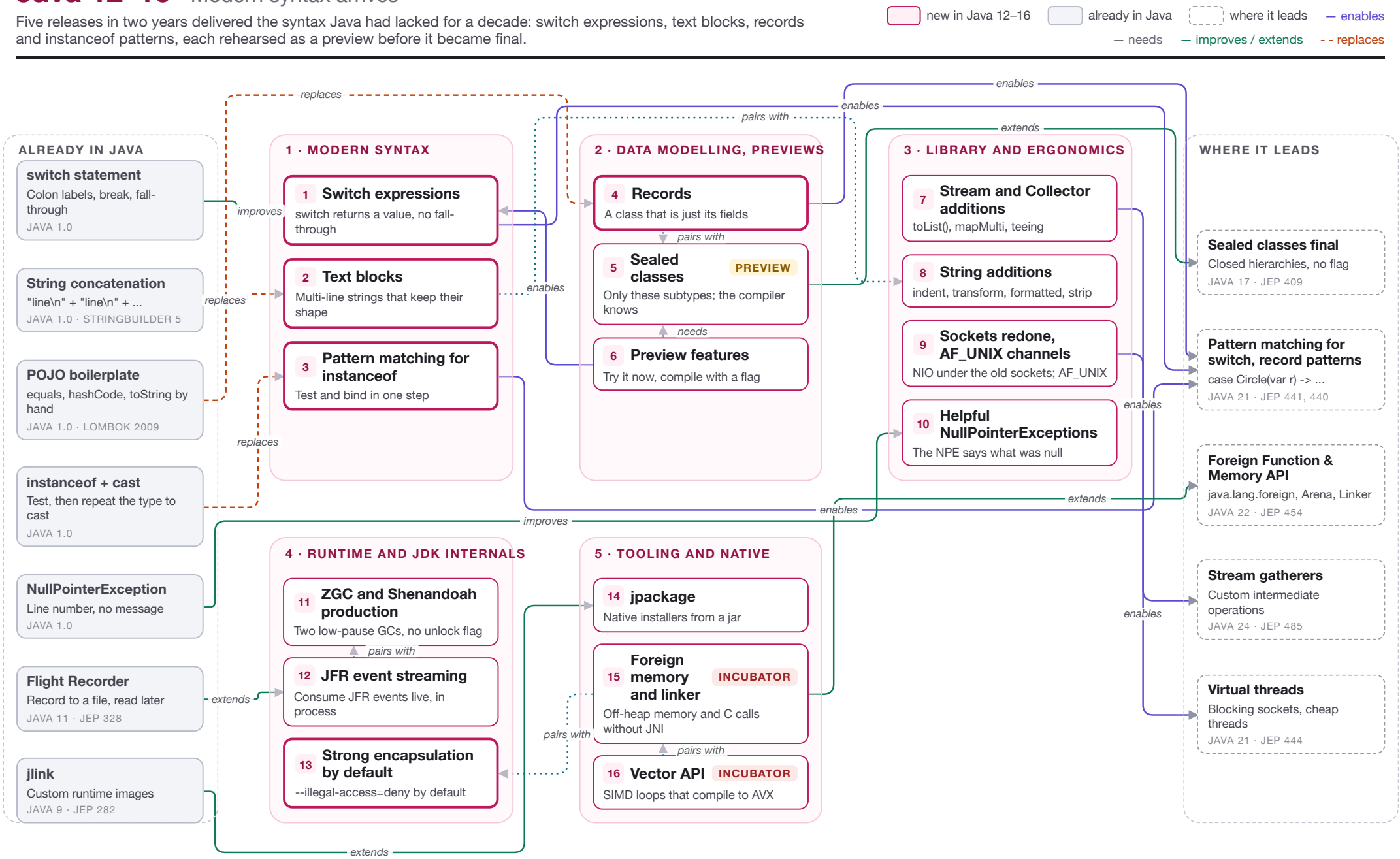


Read left to right: what Java already had → what Java 11 added, grouped by job (numbers match the cards and the cheat sheet) → what it made possible later.

CHAPTER 7 · KNOWLEDGE GRAPH · JAVA 12 – 16 · 2019 – 2021

Java 12–16 · Modern syntax arrives

Five releases in two years delivered the syntax Java had lacked for a decade: switch expressions, text blocks, records and instanceof patterns, each rehearsed as a preview before it became final.



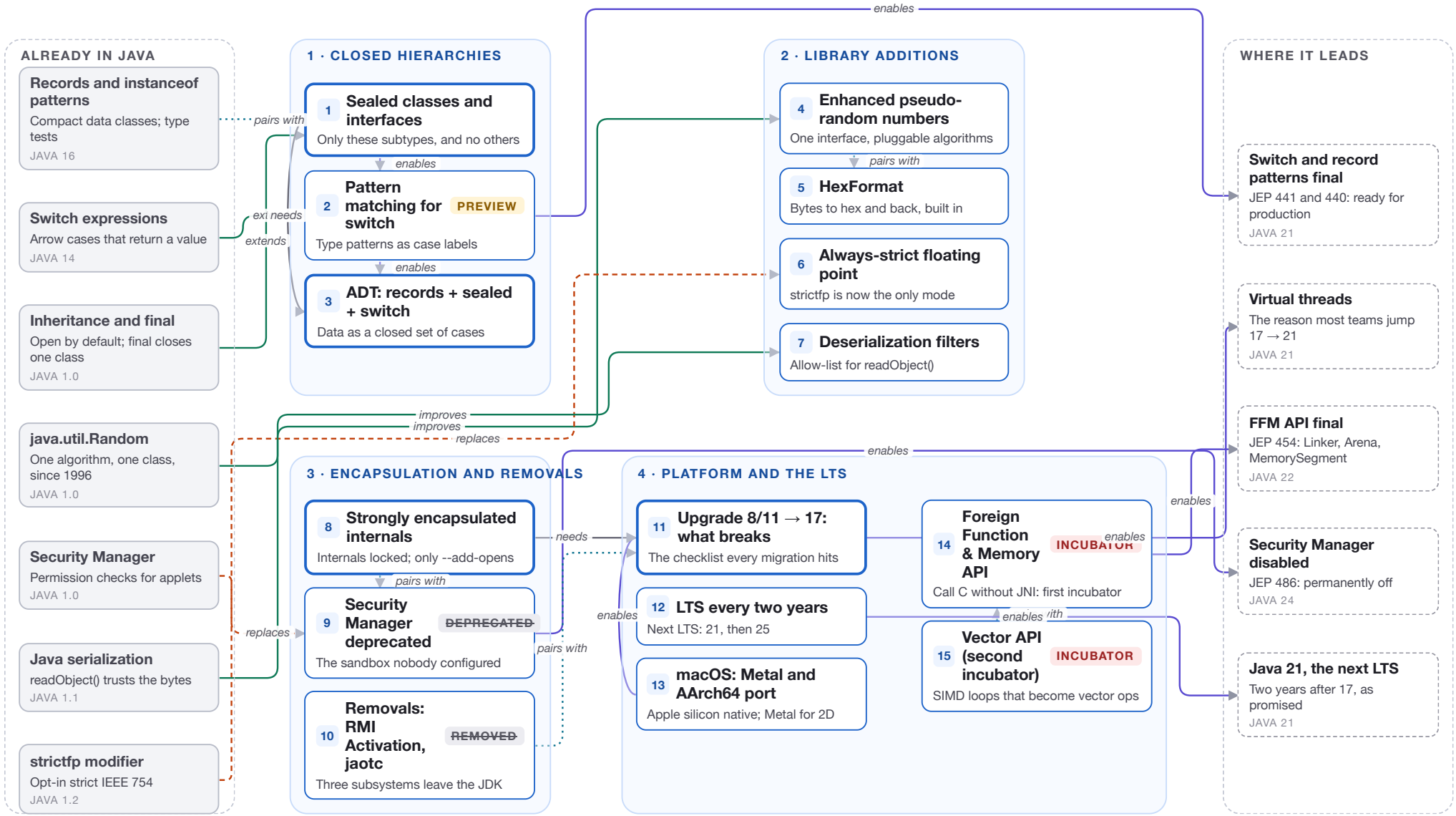
Read left to right: what Java already had → what Java 12–16 added, grouped by job (numbers match the cards and the cheat sheet) → what it made possible later.

CHAPTER 8 · KNOWLEDGE GRAPH · JAVA 17 · SEP 2021 · LTS

Java 17 · Sealed and settled

Sealed types closed the data-modelling loop that records opened, the JDK locked its internals for good, and most teams made 17 their new baseline. This chapter is both what 17 added and what the modern toolkit now assumes.

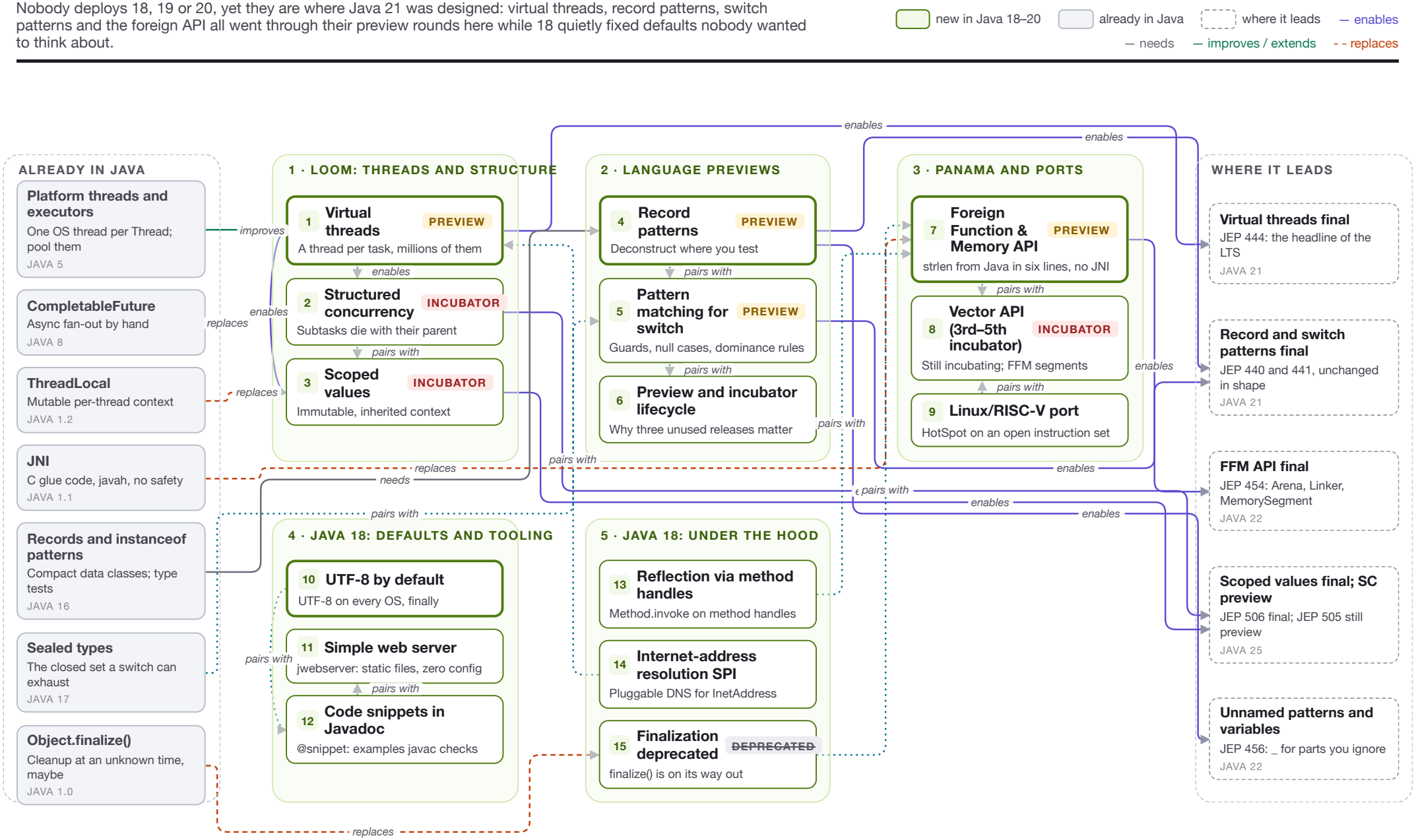
new in Java 17 · already in Java · where it leads · — enables · — needs · — improves / extends · - - replaces



CHAPTER 9 · KNOWLEDGE GRAPH · JAVA 18 – 20 · 2022 – 2023

Java 18–20 · Loom and Panama in preview

Nobody deploys 18, 19 or 20, yet they are where Java 21 was designed: virtual threads, record patterns, switch patterns and the foreign API all went through their preview rounds here while 18 quietly fixed defaults nobody wanted to think about.



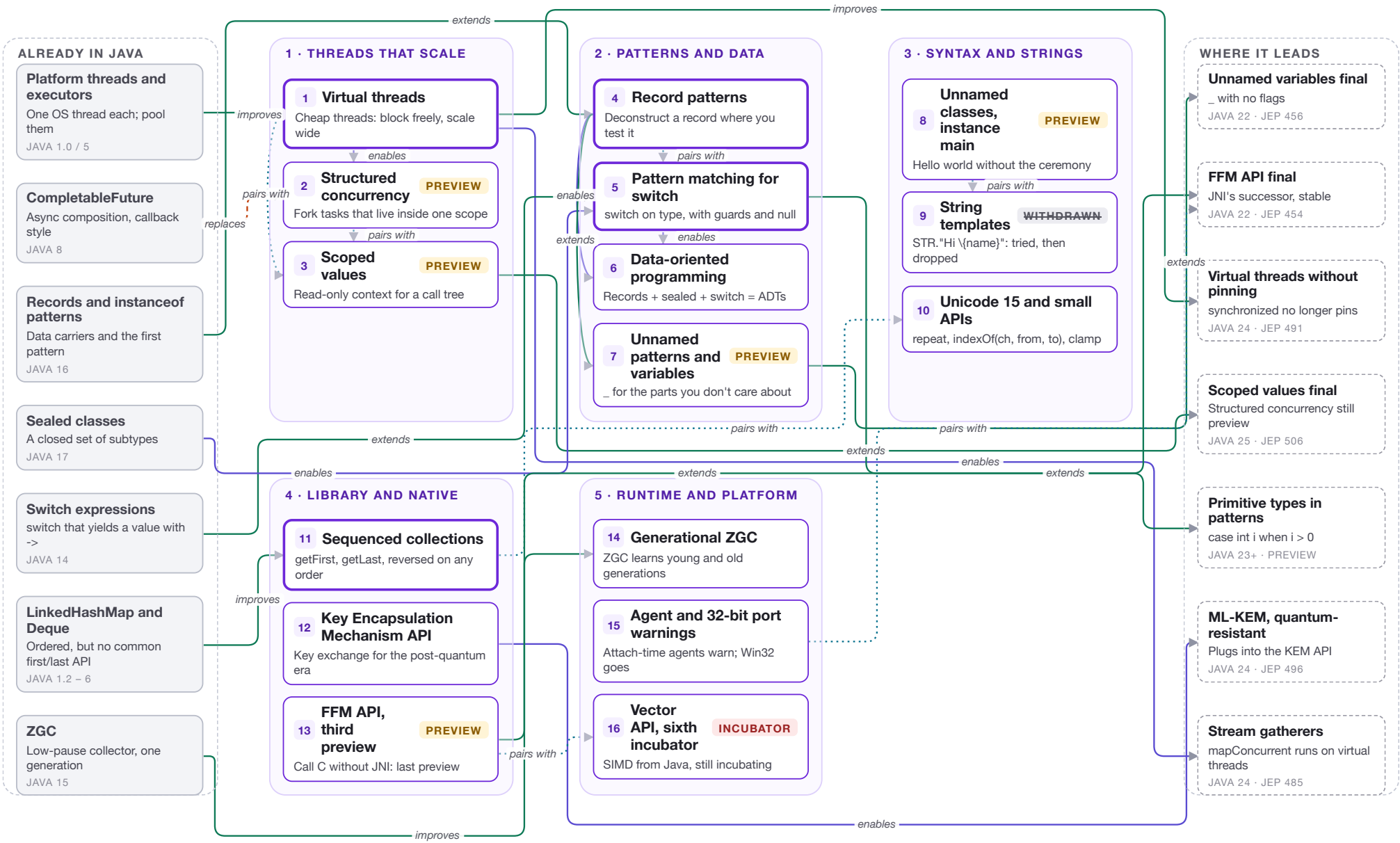
Read left to right: what Java already had → what Java 18–20 added, grouped by job (numbers match the cards and the cheat sheet) → what it made possible later.

CHAPTER 10 · KNOWLEDGE GRAPH · JAVA 21 · SEP 2023 · LTS

Java 21 · Virtual threads and patterns

Blocking code finally scales, and data finally has a shape the compiler can check. Java 21 is the LTS that makes Loom and pattern matching everyday tools.

new in Java 21 · already in Java · where it leads · — enables · — needs · — improves / extends · - - replaces

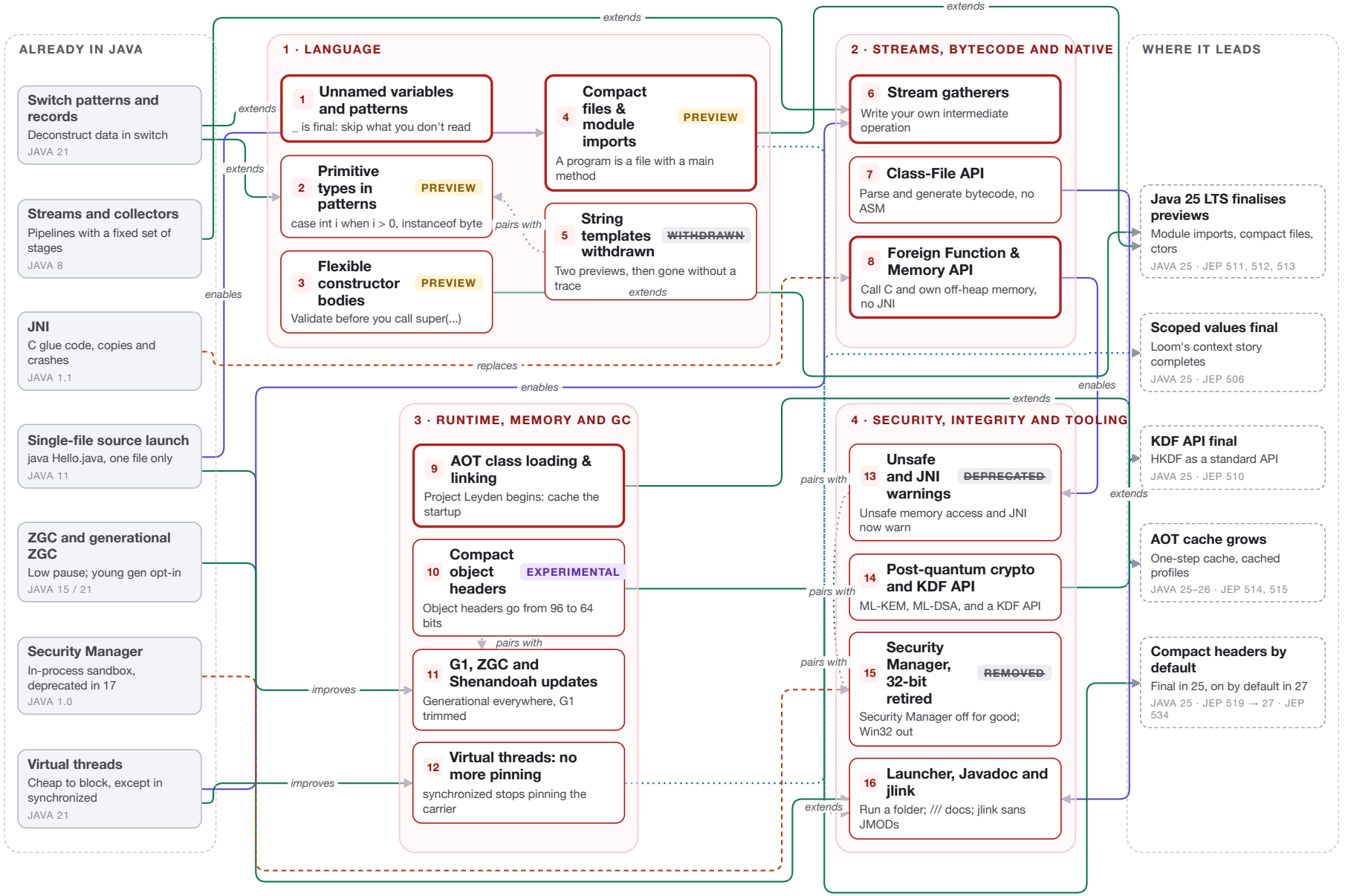


Read left to right: what Java already had → what Java 21 added, grouped by job (numbers match the cards and the cheat sheet) → what it made possible later.

Java 22–24 · Gatherers, FFM and simpler starts

Three non-LTS releases that finished Panama, gave streams a missing operation, started Project Leyden and closed the JVM's back doors. Most of what previewed here goes final in 25.

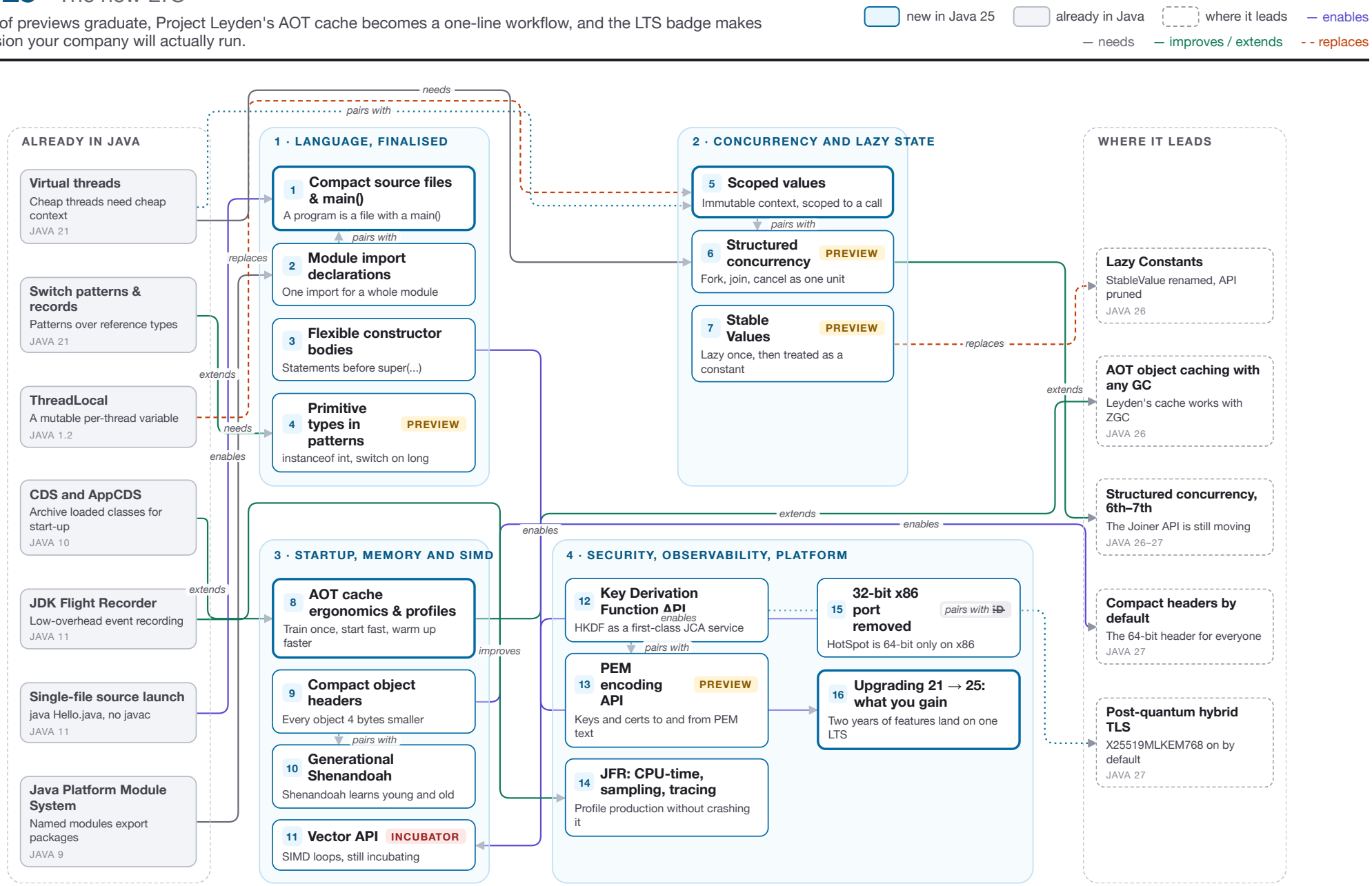
new in Java 22–24 · already in Java · where it leads · — enables · — needs · — improves / extends · - - replaces



Read left to right: what Java already had → what Java 22–24 added, grouped by job (numbers match the cards and the cheat sheet) → what it made possible later.

Java 25 · The new LTS

Two years of previews graduate, Project Leyden's AOT cache becomes a one-line workflow, and the LTS badge makes 25 the version your company will actually run.



CHAPTER 13 · KNOWLEDGE GRAPH · JAVA 26 – 27 · 2026 – 2026

Java 26–27 · Faster by default, ready for post-quantum

Two quiet-looking releases that change the defaults under every application: G1 and compact headers for everyone, HTTP/3 and a post-quantum handshake on the wire, and the warnings that say what the next LTS will refuse.

new in Java 26–27 · already in Java · where it leads · — enables · — needs · — improves / extends · - - replaces

